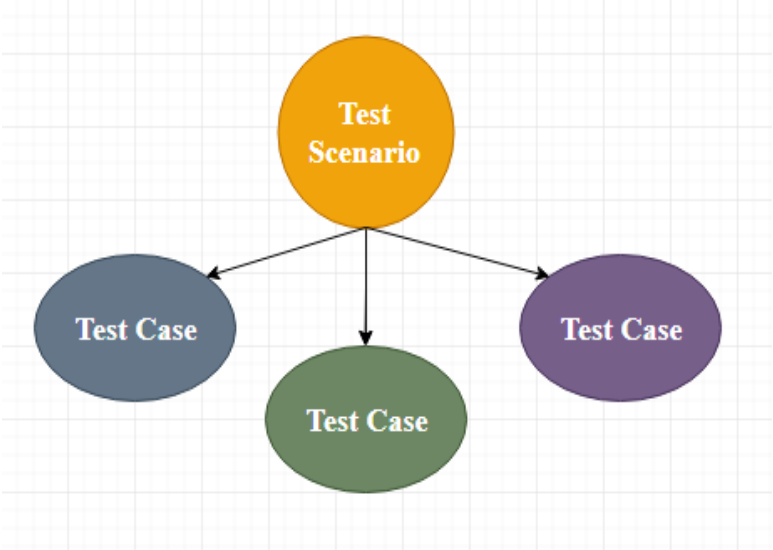
Unit 3

Building Test Cases & Plan

Test Case

- The test case is defined as a group of conditions under which a tester determines whether a software application is working as per the customer's requirements or not. Test case designing includes preconditions, case name, input conditions, and expected result. A test case is a first level action and derived from test scenarios.
- In simple terms, A Test Case contains test steps, test data, precondition, postcondition developed for specific test scenario to verify any requirement. The test case includes specific variables or conditions, using which a testing engineer can compare expected and actual results to determine whether a software product is functioning as per the requirements of the customer.



Test Case (continued…)

- It is an in-details document that contains all possible inputs (positive as well as negative) and the navigation steps, which are used for the test execution process. Writing of test cases is a one-time attempt that can be used in the future at the time of regression testing.

- Test case gives detailed information about testing strategy, testing process, preconditions, and expected output. These are executed during the testing process to check whether the software application is performing the task for that it was developed or not.

- Test case helps the tester in defect reporting by linking defect with test case ID. Detailed test case documentation works as a full proof guard for the testing team because if developer missed something, then it can be caught during execution of these full-proof test cases.

When do we write a test case?

We will write the test case when we get the following:

- When the customer gives the business needs then, the developer starts developing and says that they need 3.5 months to build this product.
- And In the meantime, the testing team will start writing the test cases.
- Once it is done, it will send it to the Test Lead for the review process.
- And when the developers finish developing the product, it is handed over to the testing team.
- The test engineers never look at the requirement while testing the product document because testing is constant and does not depends on the mood of the person rather than the quality of the test engineer.

Why we write the test cases?

We will write the test for the following reasons:

- To require consistency in the test case execution
- To make sure a better test coverage
- It depends on the process rather than on a person
- To avoid training for every new test engineer on the product

Who Writes Test Cases?

It varies from company to company. Assuming that a team contains developers and dedicated testers then it might be something like a joint effort.

Developers write Unit Tests
Developers & Testers write Integration Tests
Testers write Acceptance Tests

Format of Test case

A standard test case include following information

1. **Test Case ID**
2. **Test Case Description**
3. **Pre-Conditions**
4. **Test Steps**
5. **Test Data**
6. **Expected Result**
7. **Post Condition**
8. **Actual Result**
9. **Status**

Test Case ID:
Each test case should be represented by a unique ID. It’s good practice to follow some naming convention for better understanding and discrimination purposes.

Test Case Description:
Pick test cases properly from the test scenarios E.g
Test scenario: Verify the login of Gmail
Test case: Enter a valid username and valid password

Pre-Conditions:
Conditions that need to meet before executing the test case. Mention if any preconditions are available.
Example: Need a valid Gmail account to do login

Test Steps:
To execute test cases, you need to perform some actions. So write proper test steps. Mention all the test steps in detail and in the order how it could be executed from the end-user’s perspective.

Example:
Enter Username
Enter Password
Click Login button

Test Data:
You need proper test data to execute the test steps. So gather appropriate test data. The data which could be used an input for the test cases.
Example:
Username: rajkumar@softwaretestingmaterial.com
Password: STM
Expected Result: The result which we expect once the test cases were executed. It might be anything such as Home Page, Relevant screen, Error message, etc., **Example:** Successful login
Post Condition:
Conditions that need to achieve when the test case was successfully executed.
Example: Gmail inbox is shown
Actual Result:
The result which system shows once the test case was executed. Capture the result after the execution. Based on this result and the expected result, we set the status of the test case.
Example: Redirected to Gmail inbox
Status:
Finally set the status as Pass or Fail based on the expected result against the actual result. If the actual and expected results are the same, mention it as Passed. Else make it as Failed. If a test fails, it has to go through the bug life cycle to be fixed.
Example:
Result: Pass

General Test Case Template

Project Name:	Google Email
Module Name:	Login
Reference Document:	If any
Created by:	Rajkumar
Date of creation:	DD-MMM-YY
Date of review:	DD-MMM-YY

TEST CASE ID	TEST SCENARIO	TEST CASE	PRE-CONDITION	TEST STEPS	TEST DATA	EXPECTED RESULT	POST CONDITION	ACTUAL RESULT	STATUS (PASS/ FAIL)
TC_LOGIN_001	Verify the login of Gmail	Enter valid User Name and valid Password	1. Need a valid Gmail Account to do login	1. Enter User Name	<Valid User Name>	Successful login	Gmail inbox is shown		
				2. Enter Password	<Valid Password>				
				3. Click "Login" button					
TC_LOGIN_001	Verify the login of Gmail	Enter valid User Name and invalid Password	1. Need a valid Gmail Account to do login	1. Enter User Name	<Valid User Name>	A message "The email and password you entered don't match" is shown			
				2. Enter Password	<Invalid Password>				
				3. Click "Login" button					
TC_LOGIN_001	Verify the login of Gmail	Enter invalid User Name and valid Password	1. Need a valid Gmail Account to do login	1. Enter User Name	<Invalid User Name>	A message "The email and password you entered don't match" is shown			
				2. Enter Password	<Valid Password>				
				3. Click "Login" button					
TC_LOGIN_001	Verify the login of Gmail	Enter invalid User Name and invalid Password	1. Need a valid Gmail Account to do login	1. Enter User Name	<Invalid User Name>	A message "The email and password you entered don't match" is shown			
				2. Enter Password	<Invalid Password>				
				3. Click "Login" button					

Test case format 2

Header

Test Case Name/ID :-Release - Version - Application Name - Module

Test Case Type:-

F.T.C	I.T.C	S.T.C

Requirement Number:-

Module:-

Serverity:- Critical/Major/Minor

Status:-

Release:-

Version:-

Pre-condition:-

Test Data:-

Summary:-

F.T.C functional test case
I.T.C Integration test case
S.T.C System test case

Body

Step No.	Descri-pation	Inputs	Expected Result	Actual Reasult	Status	Comments
...	...	...	...	...	...	...
...	...	...	...	...	...	...

Footer

Author:-

Date:-

Reviewd By:-

Approved By:-

Sample Test Case Example

Test Case #	Test Case Description
1	Check response when valid email and password is entered

Test Case ID	Test Case Description	Test Steps	Test Data	Expected Results	Actual Results	Pass/Fail
TC01	Check Customer Login with valid Data	1.Go to site http://jimsindia.org 2.Enter UserId 3.Enter Password 4.Click Submit	Userid = JIMS Password = jims123	User should Login into an application	As Expected	Pass

Example 2

Test Case #	Test Case Description
2	Check response when invalid email and password is entered

TU02	Check Customer Login with invalid Data	1. Go to site http://jimsindia.org 2. Enter UserId 3. Enter Password 4. Click Submit	Userid = jims Password = 12345	User should not Login into an application	As Expected	Pass

Test data selection

- In software testing, test data selection refers to the process of choosing input data that will be used to evaluate the behavior of a software application or system. The goal of test data selection is to ensure that the software is thoroughly tested and that all potential defects or bugs are identified and addressed.
- The test data is similar to the production data used by test cases when testing software applications. It is typically collected in the test data document used by test cases and scripts. Unless test data is designed in advance, test cases may not cover all scenarios and ultimately impact software quality.

What is Test Data?

- Test data is an integral part of the testing process for a tester. It provides information to the tester to facilitate finding defects and corrective actions. Test engineers load the application with data or stress it with huge amounts of invalid data to check breakpoints and other aspects of the application's performance. While executing test cases, test engineers need to input some data into the application to get the expected output.
- Test data is usually collected into a document called a test data document that helps organize it so testers can easily access it when they run their tests.
- Testing the product with test data is essential when designing a new application. This can help determine if a product needs additional development or if it's ready to move on to further testing.

Types of test data

Test data have the following types:

1. Valid Test Data: Valid test data is the positive data used for system testing. This includes all possible inputs and conditions so that the application will run smoothly in different situations.

2. Invalid Test Data: Invalid Test Data is the negative data used to verify that a program will handle negative conditions and exceptions correctly. Some examples are:

- Null values can be used in the case of mandatory fields.
- Out of Range Values.
- Special characters are not allowed.
- Invalid data format, such as a mobile phone number with alphabetical characters in place of numbers, causes errors.

3.No Data: No data helps ensure the system will work properly if a user leaves a field blank.

4.Boundary Data: It is a pair of test data values that indicate whether an observed value falls within or outside the range of expectations.

Why is Test Data Important?

- Test data helps testers determine whether the data is ready for release or not. The correct test data is essential in deciding if the software performs as per its requirements in different scenarios. With the help of the set test data boundary in the application, test data also helps in checking for negative scenarios.

few reasons why test data is important in software testing:

1. Identify & Eliminate Bugs Early: Better test data coverage can help you identify bugs and errors early in the software testing life cycle. Identifying them early on helps in saving time and effort.
2. Enhanced Test Data Coverage: Correct test data provides clear traceability and a bird's eye view of the test cases and defect patterns.
3. Smoother Testing Cycles: Maintaining test data helps you prioritize test cases, augment your test suites, and minimize them, which means smoother and more efficient testing cycles. Effective test data can be used for both functional testing and regression testing.
4. Higher ROI(return on investment): If you reuse and maintain test data efficiently, you will have fewer defects in production and can reuse the same data set for regression testing on future projects. You will also save resources that would have been spent on creating duplicate test data for each new project and addressing defects.

How to Create Test Data?

Creating test data is crucial when it comes to delivering quality products. Here are four ways to create test data:

1. Manual Test Data Creation
2. Automated Test Data Creation
3. Backend Injection
4. Third-party tools

5. Manual test data generation is creating sample data for manual testing. One approach is to prepare a list of items used for testing, generate sample data using your QA team members or developers, and then validate that it works as expected.

- Manual test data is the most straightforward way to create test data. It is often created at the beginning of project implementation and includes all possible combinations of inputs and outputs.

How to Create Test Data?

2. Automated Test Data Creation

- Automated test data generation effectively reduces the time taken to develop, maintain, and execute tests compared to manual test data. It is performed with the help of automation testing tools like **LambdaTest** that automate the whole process from start to finish. These tools are faster and more accurate than a human-driven approach, which results in greater efficiency over time.
- During automated testing, the data that should be entered into input fields are stored in an external file. This data might be read from a database or any other data source like text or XML files, Excel sheets, and database tables.

How to Create Test Data?

3. Backend Injection

- Backend injection is one method of providing test data to a database. A tester writes relevant SQL queries, then injects them into the database to create large amounts of test data. It is easier than automated data generation methods but needs to be more accurate.
- It can be used in the following scenarios:
  - To test and debug your application without the hassle of user interaction.
  - To be able to test the accuracy of a system under a variety of circumstances.
  - To avoid the time and expense of manual data generation.

How to Create Test Data?

4. Third-party Tools

- Third-party tools can help build up your test data effectively. These tools thoroughly understand back-end applications and can pump in data like a real-time scenario. Hence, the test data is diverse and voluminous, enabling comprehensive test data coverage.
- These tools are more accurate than manual methods because they thoroughly understand the system and domain. The tools are designed so that even non-technical people can use them with little expertise in the domain. The tools' design makes them ideal for populating real-time data into the system, thus allowing users to perform necessary tests on historical data.

What is Code coverage?

- Code coverage is a measure which describes the degree of which the source code of the program has been tested. It is one form of white box testing which finds the areas of the program not exercised by a set of test cases. It also creates some test cases to increase coverage and determining a quantitative measure of code coverage.

code coverage methods

- Statement Coverage
- Decision Coverage
- Branch Coverage
- Toggle Coverage
- FSM Coverage

Statement Coverage

- Statement Coverage is a white box testing technique in which all the executable statements in the source code are executed at least once. It is used for calculation of the number of statements in source code which have been executed. The main purpose of Statement Coverage is to cover all the possible paths, lines and statements in source code.
- Statement coverage is used to derive scenario based upon the structure of the code under test.

$$\textbf{Statement Coverage} = \frac{\textit{Number of executed statements}}{\textit{Total number of statements}} \times 100$$

Generally in any software, if we look at the source code, there will be a wide variety of elements like operators, functions, looping, exceptional handlers, etc. Based on the input to the program, some of the code statements may not be executed. The goal of Statement coverage is to cover all the possible path's, line, and statement in the code.

Statement Coverage Example

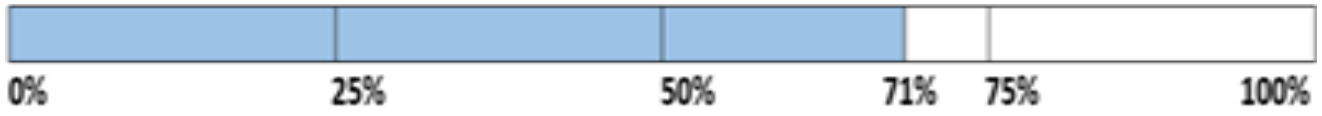
- Scenario to calculate Statement Coverage for given source code.

```
Printsum (int a, int b) {                    ------------  Printsum is a function
   int result = a+ b;
   If (result> 0)
            Print ("Positive", result)
   Else
            Print ("Negative", result)
   }                                  -----------   End of the source code
```

Scenario 1:

A=7, b=6, result is 13, result is greater than 0, so it will print "positive", 13, program will end here.

- Number of executed statements = 5, Total number of statements = 7
- Statement Coverage: 5/7 = 71%



Statement Coverage Example

- Scenario to calculate Statement Coverage for given source code.

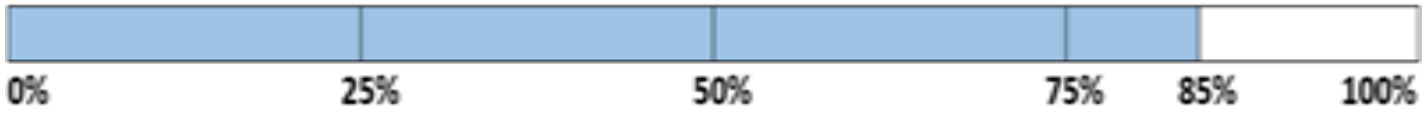
```
Printsum (int a, int b) {                    ---------- Printsum is a function
   int result = a+ b;
   If (result> 0)
            Print ("Positive", result)
   Else
            Print ("Negative", result)
   }                                  ----------   End of the source code
```

Scenario 2:

A=5, b=-9, result is -4, result is smaller than 0, so it will print "Negative", -4, program will end here.

Number of executed statements = 6, Total number of statements = 7

Statement Coverage: 6/7 = 85%



What is covered by Statement Coverage?

- Unused Statements
- Dead Code
- Unused Branches
- Missing Statements

Branch coverage

- Branch coverage is a metric used in software testing to measure the percentage of branches in the source code of a software application that have been executed during the testing process. A branch is a conditional statement in the code that can result in one of two or more possible outcomes.
- A "branch" is one of the possible execution paths the code can take after a decision statement—e.g., an **if** statement—gets evaluated.
- Branch coverage is an important metric in that it can help a team or organization assess whether an application has been tested to completion. A low branch coverage shows that there are scenarios in the application lacking testing.

Branch coverage

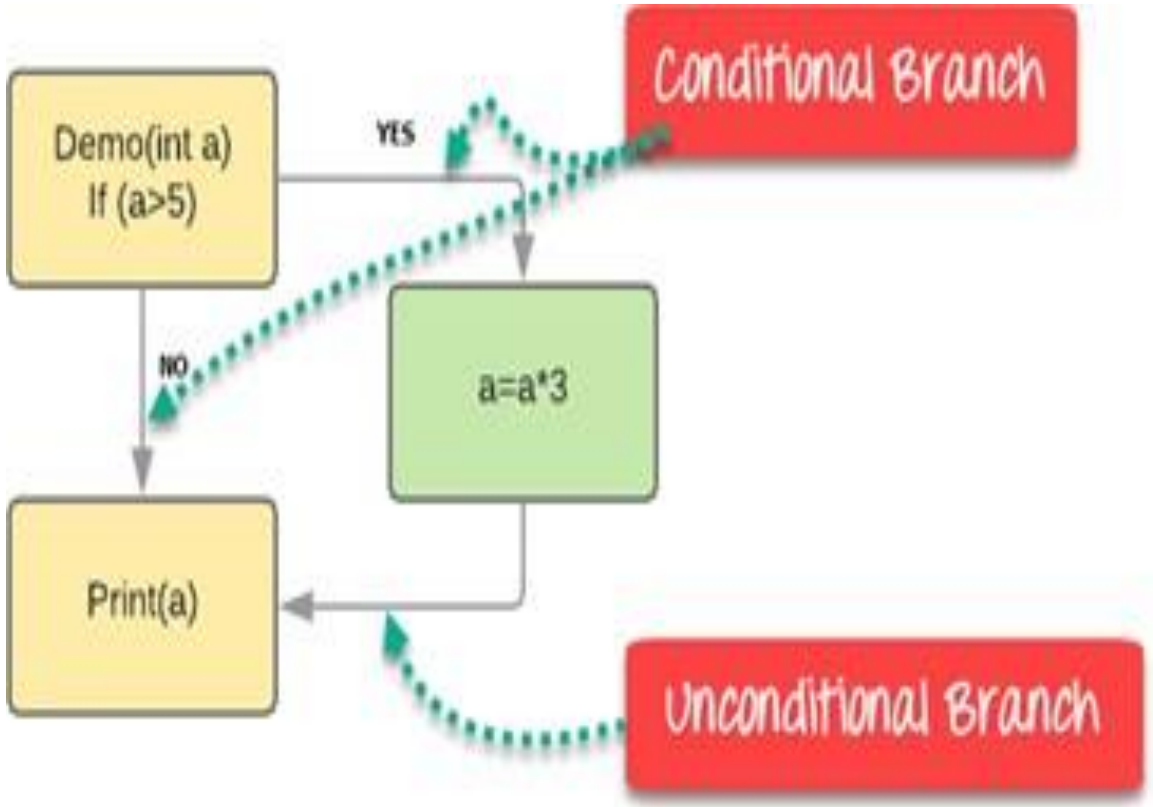
- **Branch Coverage** is a white box testing method in which every outcome from a code module(statement or loop) is tested. The purpose of branch coverage is to ensure that each decision condition from every branch is executed at least once. It helps to measure fractions of independent code segments and to find out sections having no branches.
- For example, if the outcomes are binary, you need to test both True and False outcomes.
- **The formula to calculate Branch Coverage:**
- $$Branch\ Coverage = \frac{Number\ of\ Executed\ Branches}{Total\ Number\ of\ Branches}$$
.

Example of Branch Coverage

To learn branch coverage, let's consider the same example used earlier

Consider the following code-

```
Demo(int a) {
    If (a> 5)
        a=a*3
    Print (a)
  }
```



Branch Coverage will consider unconditional branch as well

Test Case	Value of A	Output	Decision Coverage	Branch Coverage
1	2	2	50%	**33%**
2	6	18	50%	**67%**

Advantages of Branch coverage:

Branch coverage Testing offers the following advantages:

- Allows you to validate-all the branches in the code
- Helps you to ensure that no branched lead to any abnormality of the program's operation
- Branch coverage method removes issues which happen because of statement coverage testing
- Allows you to find those areas which are not tested by other testing methods
- It allows you to find a quantitative measure of code coverage
- Branch coverage ignores branches inside the Boolean expressions
- Solve by urself

Solve the following for branch coverage

- Read A
- Read B
- IF A+B > 10 THEN
- Print "A+B is Large"
- ENDIF
- If A > 5 THEN
- Print "A Large"
- ENDIF

Precondition & post condition

- The set of conditions that must be in place before testing can start are called **preconditions**. Pre-condition is a statement or set of statements that outline a condition that should be true when an action is called. The precondition statement indicates what must be true before the function is called.
- The preconditions for a test case include the state a system and its environment must be before a specific test can be run. In other words, preconditions specify the setup needed for a test case to be executed successfully.
- Example:
- To identify the square root of a number, the precondition is that the number should be greater than zero. Upon executing the pre condition, the square root of the number is displayed on the console.

Precondition & post condition

Precondition…….

- There are many possible types of settings and conditions that can be preconditions to a given test case but, by and large, preconditions fall under one of the follow three classes:
- A previously executed step in the testing sequence.
- The outcome of a previous step in the testing sequence.
- The availability of existing data needed to run the test case.
- Preconditions are generally listed under a separate heading in the written test case. If no preconditions are necessary then this heading would be left empty, but otherwise you should read it carefully to note any steps that need to be carried out before you can execute the actual test.

Precondition & post condition

preconditions that are essential for successful test execution

#1 – A Properly Configured Test Environment

- The test environment must be configured correctly in order for the results to be reliable. With a properly configured test system, many problems can be avoided and a smoother, more efficient testing process achieved.

#2 – A Process For Managing Identified Defects

- Testers must have a way, usually a bug tracking software, for handling any errors that crop up when running test cases.
- Additionally, standard tools used in popular agile methodologies like Scrum, e.g. agile boards and burndown charts, should be integrated in the software and made available to testing teams at affordable and flexible terms. Integration with third-party testing tools would also be a plus.

Precondition & post condition

Post condition

- Post Condition is a statement or set of statements describing the outcome of an action if true when the operation has completed its task.
- The Post Conditions statement indicates what will be true when the action finishes its task.

A postcondition is a condition or predicate that must always be true just after the execution of some section of code or after an operation in a formal specification. Postconditions are sometimes tested using assertions within the code itself. Often, postconditions are simply included in the documentation of the affected section of code.

For example: The result of a factorial is always an integer and greater than or equal to 1. So a program that calculates the factorial of an input number would have postconditions that the result after the calculation be an integer and that it be greater than or equal to 1.

Precondition & post condition

One more example

We will take an eg. of Login Page
Precondition (prerequsite) : S/w should be install on system,login page should be available on desktop
postcondition : login successful.

Please check a complete test case example ,visit

https://softwaretestingfundamentals.com/test-case/

Test Schedule & check points

- In software testing, a test schedule is a document or plan that outlines the scope, timeline, and resources required for testing a software application. It helps ensure that testing is completed within a specific time frame and helps the testing team prioritize their efforts.
- Checkpoints, on the other hand, refer to specific milestones or stages within the testing process. They represent specific points in time where the testing team evaluates the progress of testing and determines whether the software meets the required quality standards.
- In addition, checkpoints can also refer to the process of saving intermediate results during testing, known as checkpoints or check pointing. Check pointing allows testers to save the current state of the testing process in case of a system failure or other unexpected event that could cause data loss. This way, testers can resume testing from the saved checkpoint rather than starting from the beginning, which can save time and resources.

End Of Unit 3